Reviewer Pack — Minimal Geometric Complexity of Boolean Functions and Commun…

Entry 12133d6a57ba · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 01:33:33 UTC

Minimal Geometric Complexity of Boolean Functions and Communication Rank Variance

Entry ID: `12133d6a57ba` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 01:33:33 UTC

1. Conjecture

Field A (mathematical branch): Geometric Complexity Theory (GCT) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

> For every n-ary boolean function f, the geometric complexity GC(f) is linearly correlated with the variance in communication rank CR(f), such that Var(CR(f)) = Θ(GC(f)^2).

Rationale (proposer's reasoning):

> Geometric Complexity Theory (GCT) has shown potential in characterizing complexity of mathematical structures through algebraic geometry. This conjecture links GCT to communication complexity by proposing a geometric measure that quantifies the variance in communication rank, potentially revealing new insights into the structure of boolean functions.

Taxonomy category: `GCT_TO_COMMUNICATION_RANK_VARIANCE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8bf4b02fdb9373a5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between GC(f) and Var(CR(f)) across all seeds exceeds 0.5, with a p-value < 0.01.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `intitle:(Minimal Geometric Complexity of Boolean Functions OR GCT) AND intitle:(Communication Rank Variance OR Matrix Rank Variance)` - `all:geometric complexity theory site:arxiv.org AND all:communication rank variance site:arxiv.org` - `title:(Boolean functions AND Communication rank variance) AND title:(Geometric Complexity Theory OR GCT)`

Top relevant hits considered: - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1912.12844v1] Variance Reduced Local SGD with Lower Communication Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=-9, elapsed=153.5s

5.1 Generated Python source

```python
import random
import math
from collections import Counter

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_rank(f):
        n = int(math.log2(len(f)))
        count = Counter()
        for i in range(2**n):
            for j in range(i+1, 2**n):
                if f[i] != f[j]:
                    count[(i, j)] += 1
        return len(count)

    def geometric_complexity(f):
        n = int(math.log2(len(f)))
        count = Counter()
        total = 0
        for i in range(2**n):
            for j in range(i+1, 2**n):
                if f[i] != f[j]:
                    count[(i, j)] += 1
                    total += 1
        return sum(count[i] * math.log2((count[i]+1)/total) for i in count)
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    gc_sum = 0
    cr_variance = 0
    instances_tested = 0

    for n in n_values:
        f = generate_boolean_function(n)
        gc = geometric_complexity(f)
        cr = communication_rank(f)

        if cr == 0:
            continue

        gc_sum += gc
        cr_variance += (cr - gc)**2
        instances_tested += 1

    if instances_tested < 30:
        return {
            "metric_name": "geometric_complexity",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "not_enough_instances"
        }

    gc_mean = gc_sum / instances_tested
    cr_variance /= instances_tested

    return {
        "metric_name": "geometric_complexity",
        "metric_value": cr_variance,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": cr_variance >= gc_mean**2 * 0.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
```

```
        counterexample = "not_enough_support"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
(empty)
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means that the Pearson correlation coefficient and p-value could not be calculated. | next: Re-run the test to ensure it completes successfully and produces the required data for calculating the Pearson correlation coefficient and p-value.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12526
2	propose	ollama_remote	glm4:latest	0	0	12802
3	propose	ollama_remote	glm4:latest	0	0	12475
4	propose	ollama_remote	glm4:latest	0	0	15124
5	preregistration	ollama_remote	glm4:latest	0	0	16727
6	novelty	ollama_remote	glm4:latest	0	0	8361
7	novelty	ollama_remote	glm4:latest	0	0	9656
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24432
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11918
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10497
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8835
12	judge	ollama_remote	glm4:latest	0	0	61732

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 205086 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/12133d6a57ba.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/12133d6a57ba.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/12133d6a57ba.tar.gz` (if generated)